

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

```
import re
```

```
text = "My email is student123@gmail.com and my phone number is 9876543210."
```

```
email = re.search(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)
```

```
phone = re.search(r'\b\d{10}\b', text)
```

```
words = re.findall(r'\b\w+\b', text)
```

```
print("Email:", email.group())
```

```
print("Phone:", phone.group())
```

```
print("Words:", words)
```

```
import re

text = "My email is student123@gmail.com and my phone number is 9876543210."

# Search for email
email = re.search(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', text)

# Search for phone number
phone = re.search(r'\b\d{10}\b', text)

# Find all words
words = re.findall(r'\b\w+\b', text)

print("Email:", email.group())
print("Phone:", phone.group())
print("Words:", words)
```

```
*** Email: student123@gmail.com
Phone: 9876543210
Words: ['My', 'email', 'is', 'student123', 'gmail', 'com', 'and', 'my', 'phone', 'number', 'is', '9876543210']
```

2. Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

```
def fsa(string):
```

```
state = 0
```

```
for ch in string:
```

```
    if state == 0:
```

```
        if ch == 'a':
```

```
            state = 1
```

```
        else:
```

```
            state = 0
```

```
    elif state == 1:
```

```
        if ch == 'a':
```

```
            state = 1
```

```
        elif ch == 'b':
```

```
            state = 2
```

```
        else:
```

```
            state = 0
```

```
    elif state == 2:
```

```
        if ch == 'a':
```

```
            state = 1
```

```
        else:
```

```
            state = 0
```

```
if state == 2:
```

```
    print("Accepted")
```

```
else:
```

```
    print("Rejected")
```

```
string = input("Enter a string: ")
```

```
fsa(string)
```

```
def fsa(string):
    state = 0

    for ch in string:
        if state == 0:
            if ch == 'a':
                state = 1
            else:
                state = 0

        elif state == 1:
            if ch == 'a':
                state = 1
            elif ch == 'b':
                state = 2
            else:
                state = 0

        elif state == 2:
            if ch == 'a':
                state = 1
            else:
                state = 0

    if state == 2:
        print("Accepted")
    else:
        print("Rejected")

string = input("Enter a string: ")
fsa(string)
```

*** Enter a string: madhu
Rejected

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

```
import nltk

from nltk.stem import PorterStemmer

from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

stemmer = PorterStemmer()

lemmatizer = WordNetLemmatizer()

words = ["running", "studies", "better", "playing", "cars"]

print("Word\t\tStem\t\tLemma")
```

for word in words:

 stem = stemmer.stem(word)

 lemma = lemmatizer.lemmatize(word)

 print(f"{word}\t\t{stem}\t\t{lemma}")

```
import nltk
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

words = ["running", "studies", "better", "playing", "cars"]

print("Word\t\tStem\t\tLemma")

for word in words:
    stem = stemmer.stem(word)
    lemma = lemmatizer.lemmatize(word)
    print(f"{word}\t\t{stem}\t\t{lemma}")
```

Word	Stem	Lemma
running	run	running
studies	studi	study
better	better	better
playing	play	playing
cars	car	car

[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!

4. Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

def generate_plural(noun):

"""

Simulates a Finite-State Machine (FSM) for morphological parsing
 to convert singular English nouns to their plural forms.

"""

State q0: Start / Input Inspection

if noun.endswith(("s", "x", "z", "ch", "sh")):

Transition to State q1: Append "es" rule

plural = noun + "es"

```
elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
```

```
    # Transition to State q2: Consonant + 'y' rule (replace 'y' with 'ies')
```

```
    plural = noun[:-1] + "ies"
```

```
else:
```

```
    # Transition to State q3: Default rule (append "s")
```

```
    plural = noun + "s"
```

```
return plural
```

```
# Test words representing different state transitions
```

```
words = ["cat", "dog", "bus", "box", "church", "dish", "baby", "city", "toy", "book"]
```

```
print("Singular\tPlural")
```

```
print("-----")
```

```
for word in words:
```

```
    print(f'{word:10}\t{generate_plural(word)}')
```

```
def generate_plural(noun):
    """
    Simulates a Finite-State Machine (FSM) for morphological parsing
    to convert singular English nouns to their plural forms.
    """
    # State q0: Start / Input Inspection
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        # Transition to State q1: Append "es" rule
        plural = noun + "es"

    elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
        # Transition to State q2: Consonant + 'y' rule (replace 'y' with 'ies')
        plural = noun[:-1] + "ies"

    else:
        # Transition to State q3: Default rule (append "s")
        plural = noun + "s"

    return plural

# Test words representing different state transitions
words = ["cat", "dog", "bus", "box", "church", "dish", "baby", "city", "toy", "book"]

print("Singular\tPlural")
print("-----")

for word in words:
    print(f'{word:10}\t{generate_plural(word)}')
```

Singular	Plural
cat	cats
dog	dogs
bus	buses
box	boxes
church	churches
dish	dishes
baby	babies
city	cities
toy	toys
book	books

5. Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

```
import nltk

from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = [
    "running", "playing", "studies", "studying",
    "connected", "connection", "happiness",
    "flying", "wolves", "cars"
]

print("Original Word\t\tStemmed Word")
print("-" * 40)

# Perform stemming
for word in words:
    stem = stemmer.stem(word)
    print(f"{word:20}{stem}")
```

```
import nltk
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = [
    "running", "playing", "studies", "studying",
    "connected", "connection", "happiness",
    "flying", "wolves", "cars"
]

print("Original Word\t\tStemmed Word")
print("-" * 40)

for word in words:
    stem = stemmer.stem(word)
    print(f"{word:20}{stem}")
```

Original Word	Stemmed Word
cars	car

6. Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

```
import random

from collections import defaultdict, Counter

def build_bigram_model(corpus):
    # Tokenize text into lowercase words
    words = corpus.lower().split()

    # Create a mapping: current_word -> Counter of next_words
    model = defaultdict(Counter)

    for w1, w2 in zip(words[:-1], words[1:]):
        model[w1][w2] += 1

    return model

def generate_text(model, start_word, length=15):
    current_word = start_word.lower()
    output = [current_word]

    for _ in range(length - 1):
        if current_word not in model:
            break # Stop if the current word has no known followers

        # Weighted random selection based on frequency
        next_words = list(model[current_word].keys())
        counts = list(model[current_word].values())

        current_word = random.choices(next_words, weights=counts)[0]
```

```
output.append(current_word)
```

```
return " ".join(output)
```

```
# Example Usage
```

```
training_text = """
```

```
The boys are running in the park. The girls are writing letters.
```

```
The children played games in the garden while the boys played football.
```

```
"""
```

```
# Build the bigram model
```

```
bigram_model = build_bigram_model(training_text)
```

```
# Generate a sequence starting with "the"
```

```
generated_text = generate_text(bigram_model, start_word="the", length=10)
```

```
print("Generated Text:", generated_text)
```

```
import random
from collections import defaultdict, Counter

def build_bigram_model(corpus):
    # Tokenize text into lowercase words
    words = corpus.lower().split()

    # Create a mapping: current_word -> Counter of next_words
    model = defaultdict(Counter)

    for w1, w2 in zip(words[:-1], words[1:]):
        model[w1][w2] += 1

    return model

def generate_text(model, start_word, length=15):
    current_word = start_word.lower()
    output = [current_word]

    for _ in range(length - 1):
        if current_word not in model:
            break # Stop if the current word has no known followers

        # Weighted random selection based on frequency
        next_words = list(model[current_word].keys())
        counts = list(model[current_word].values())

        current_word = random.choices(next_words, weights=counts)[0]
        output.append(current_word)

    return " ".join(output)

# Example Usage
training_text = """
The boys are running in the park. The girls are writing letters.
The children played games in the garden while the boys played football.
"""

# Build the bigram model
bigram_model = build_bigram_model(training_text)

# Generate a sequence starting with "the"
generated_text = generate_text(bigram_model, start_word="the", length=10)
print("Generated Text:", generated_text)
```

```
Generated Text: the boys played games in the boys played games in
```